

GameForge

GameForge is a cyber-themed Python CLI for game library management with sentiment analysis, financial tracking, and smart lore-based recommendations.

Project Title: GameForge

Subtitle: Cyber-Themed Video Game Library & Analytics Platform

Author: Aditya Amol Dhepe

Date: 18-June-2026

Repository: https://github.com/ar3misByte/24BEL0044_Aditya.git

Tech Stack: Python 3.10+ | MongoDB | pymongo | mongomock | Custom Terminal UI

```
aditya@oasis:~/Documents/24BEL0844_Aditya$ source /home/aditya/Documents/24BEL0844_Aditya/.venv/bin/activate
(.venv) aditya@oasis:~/Documents/24BEL0844_Aditya$ /home/aditya/Documents/24BEL0844_Aditya/.venv/bin/python /home/aditya/Documents/24BEL0844_Aditya/GameForge/main.py
```

GAMEFORGE

Cyber terminal game tracking system

1. Add user
2. List users
3. Update user
4. Delete user
5. Add game
6. List games
7. Update game
8. Delete game
9. Track a game
10. Update tracked game
11. Delete tracked game
12. View user library
13. Inspect game HUD
14. Lore-link suggestions
15. Search games
16. View user stats
17. List all library entries
18. Exit

Choose an option:

Table of Contents

1. Executive Summary
2. Abstract
3. Project Overview
4. Problem Statement
5. Solution Overview
6. Key Features
7. Technical Architecture
8. Project Structure
9. Database Design
10. Functional Operations
11. Core Algorithms
12. User Interface Design
13. Installation and Setup
14. Configuration
15. Seed Data
16. Analytics and Reporting
17. Strengths and Innovations
18. Error Handling and Edge Cases
19. Screenshots
20. Future Enhancements
21. Conclusion
22. Appendix

1. Executive Summary

GameForge is a cyber-themed terminal application for comprehensive video game library management, analytics, and discovery. It helps users organize their game collections, track gameplay activity, record sentiment and ratings, analyze financial value through cost-per-hour metrics, identify neglected backlog through dust detection, and discover related games using a lore-link recommendation engine.

Built with Python 3.10+ and MongoDB, GameForge uses a modular service-oriented CLI architecture and a custom ANSI-rendered terminal interface. It supports complete CRUD functionality across users, games, and library entries, while also offering advanced search, gameplay dashboards, and recommendation features.

GameForge addresses the common problem of fragmented game tracking by providing a unified, data-driven platform that combines metadata management, analytics, and personalized game discovery in one immersive CLI experience.

2. Abstract

The gaming library management landscape often lacks unified tools for tracking not only owned and played games, but also emotional engagement, financial value, playstyle preferences, and narrative relationships between titles.

GameForge addresses this gap through a feature-rich Python console application that enables users to:

- Organize personal game libraries with rich metadata
- Track gameplay metrics such as hours played, completion percentage, and achievements
- Record emotional sentiment and numeric scores
- Compute financial analytics such as cost per hour
- Identify neglected games through dust detection
- Discover related games using shared lore tags, genres, and playstyles
- Maintain player profiles with platform and genre preferences

The application stores all data in MongoDB, benefiting from a flexible schema that accommodates varied game metadata. It also supports mongomock fallback, enabling offline development and testing when MongoDB is unavailable.

3. Project Overview

What is GameForge?

GameForge is a video game library and analytics platform designed as a cyberpunk-inspired CLI tool. It combines collection management with data-driven insights so users can manage what they own, what they play, how they feel about games, and whether those games were worth the money.

Primary Goals

- Track games owned and played with rich metadata
- Analyze gameplay habits through hours, sentiment, and completion statistics
- Evaluate gaming investment through cost-per-hour calculations
- Discover similar games using lore tags, genres, and playstyles
- Identify backlog and neglected games through inactivity detection

4. Problem Statement

Modern gamers often manage libraries across multiple platforms, genres, and storefronts. Existing tracking methods are fragmented and typically fail to provide:

- Unified collection management
- Emotional sentiment tracking
- Financial efficiency metrics
- Intelligent recommendations based on narrative and mechanical similarity
- Clear visibility into neglected backlog

As a result, players struggle to understand their gaming habits, evaluate their purchases, and discover related titles efficiently.

5. Solution Overview

GameForge provides a unified CLI platform that solves these problems by combining:

- Centralized library tracking
- User and game profile management
- Sentiment and rating capture
- Cost-per-hour financial analysis
- Dust detection for inactive games
- Smart lore-based recommendations
- Advanced search across multiple metadata fields

This turns a basic game tracker into a broader gameplay intelligence system.

6. Key Features

Core Highlights

- Full terminal workflow with HUD-like interface
- Complete CRUD operations for users, games, and library entries
- Multi-dimensional game cataloging
- Sentiment tracking with emotional state and score
- Financial analysis using cost-per-hour
- Dust detection for backlog management
- Lore-link recommendation engine
- Advanced multi-field search
- User statistics dashboard
- MongoDB with mongomock fallback
- Seed script with demo-ready data

7. Technical Architecture

Technology Stack

Component	Technology
Language	Python 3.10+
Database	MongoDB
Fallback Database	mongomock
Driver	pymongo
UI Framework	Custom ANSI terminal rendering
Architecture	Service-oriented CLI application

Architectural Style

GameForge follows a modular service-oriented CLI architecture. Responsibilities are separated into:

- Entry point and menu dispatching
- Database configuration
- Data models
- Business logic and analytics
- Interactive CLI flow
- Terminal rendering utilities

This separation improves maintainability, readability, and testability.

GameForge/

```
|— main.py
|— seed.py
|— config/
|   |— database.py
|— models/
|   |— user_model.py
|   |— game_model.py
|— services/
|   |— library_service.py
|   |— cli.py
|— utils/
|   |— terminal_ui.py
```

File Responsibilities

File	Responsibility
main.py	Application entry point and menu dispatcher
seed.py	Seeds demo users, games, and library entries
config/database.py	Database connection and fallback setup
models/user_model.py	User CRUD model and validation
models/game_model.py	Game and library entry models
services/library_service.py	Business logic, analytics, and recommendations
services/cli.py	Interactive menu and input handling
utils/terminal_ui.py	HUD rendering, box layouts, and terminal formatting

9. Database Design

GameForge uses three primary collections:

1. users
2. games
3. library

9.1 Users Collection

```
{
  "_id": ObjectId,
  "username": str,
  "email": str,
  "favorite_platform": str,
  "favorite_genres": [str],
  "playstyles": [str],
  "bio": str,
  "created_at": datetime,
  "updated_at": datetime
}
```

9.2 Games Collection

```
{
  "_id": ObjectId,
  "title": str,
  "developer": str,
  "release_year": int,
  "platform": str,
  "genre": [str],
  "playstyles": [str],
  "lore_tags": [str],
  "price_paid": float,
  "description": str,
  "added_at": datetime,
```

```
"created_at": datetime,  
"updated_at": datetime  
}
```

9.3 Library Collection

```
{  
  "_id": ObjectId,  
  "user_id": ObjectId,  
  "game_id": ObjectId,  
  "hours_played": float,  
  "status": str,  
  "sentiment_state": str,  
  "sentiment_score": int,  
  "rating": int,  
  "comment": str,  
  "achievements_unlocked": int,  
  "completion_percentage": float,  
  "last_played_at": datetime,  
  "created_at": datetime,  
  "updated_at": datetime  
}
```

Derived Runtime Fields

The following values are computed dynamically at runtime:

```
{  
  "cost_per_hour": float,  
  "financial_value": str,  
  "dust_state": str,  
  "dust_days": int,  
  "dust_score": float  
}
```

Why MongoDB?

- Flexible schema for diverse game metadata
- Natural fit for JSON-like Python structures
- Good support for evolving fields and experimentation
- Suitable for scaling and complex filtering
- Easier handling of array-based metadata like genres, tags, and playstyles

10. Functional Operations

GameForge provides 20 major operations grouped into four categories.

10.1 User Management

1. Create user with preferences
2. List all users
3. Update user profile
4. Delete user
5. Resolve user by ID or username

10.2 Game Management

6. Create game with full metadata
7. List games with optional filters
8. Update game details
9. Delete game
10. Inspect game HUD

10.3 Library Tracking

11. Track a game for a user
12. Update tracked entry
13. Delete tracked entry
14. View a single user's library
15. List all library entries

10.4 Analytics and Discovery

16. View user stats dashboard
17. Lore-link recommendations
18. Advanced game search
19. Dust detection report
20. Financial analysis report

11. Core Algorithms

11.1 Cost-Per-Hour Calculation

GameForge evaluates value-for-money using:

$$CPH = \frac{price_paid}{hours_played}$$

This metric estimates how much each hour of entertainment costs.

Use Cases

- Identify games that delivered strong value
- Compare purchases objectively
- Highlight underused expensive titles

11.2 Financial Verdict Classification

Based on cost-per-hour and gameplay activity, the system assigns verdicts such as:

- HIGH VALUE: CPH below average and hours played above 20
- EXPENSIVE: CPH above average and hours played below 5
- GOOD VALUE: Balanced cost and playtime
- BUDGET FRIENDLY: Very low cost-per-hour

11.3 Dust State Detection

The dust detection algorithm identifies neglected games using inactivity.

`days_inactive=today-last_played_at`

If inactivity exceeds 180 days:

`dust_state=DUSTY`

Otherwise:

`dust_state=ACTIVE`

A normalized `dust_score` from 0 to 100 estimates relative inactivity severity.

Purpose

- Detect backlog buildup
- Help prioritize older unfinished games
- Surface forgotten purchases

11.4 Lore-Link Recommendation Engine

The recommendation engine compares games based on shared:

- lore tags
- genres
- playstyles

A simplified similarity concept is:

$$\textit{similarity_score} = \frac{\textit{shared_traits}}{\textit{total_traits}}$$

Games above a selected threshold are returned and ranked by score.

Benefits

- Goes beyond simple genre matching
- Supports thematic discovery
- Helps users find narratively similar experiences

11.5 Multi-Field Search

GameForge supports case-insensitive partial matching across:

- title
- developer
- description
- genre
- playstyles
- lore_tags

This allows flexible search from both mechanical and narrative angles.

12. User Interface Design

UI Style

GameForge uses a cyberpunk terminal UI built with custom ANSI rendering.

Visual Features

- Cyan, magenta, green, and red color themes
- Box-drawing layouts
- HUD-like detailed displays
- Table formatting for libraries and analytics
- Interactive numbered menus
- User-friendly validation messages

Example Menu

```
┌────────────────────────────────────────────────────────────────────────────────┐
│  GAMEFORGE - GAME LIBRARY MANAGER  │
│  Cyber-Themed CLI Application  │
└────────────────────────────────────────────────────────────────────────────────┘
```

1. Add user
2. List users
3. Update user
4. Delete user
5. Add game
6. List games
7. Update game
8. Delete game
9. Track a game
10. Update tracked game
11. Delete tracked game
12. View user library
13. Inspect game HUD
14. Lore-link suggestions
15. Search games
16. View user stats
17. List all library entries
18. Exit

13. Installation and Setup

Prerequisites

- Python 3.10 or newer
- MongoDB 4.4+ recommended
- Or mongomock enabled for fallback

Installation Steps

```
pip install -r requirements.txt
```

```
mongod
```

```
python seed.py
```

```
python main.py
```

Alternative Execution Flow

```
python -m pip install -r requirements.txt
```

```
python seed.py
```

```
python main.py
```

14. Configuration

Environment Variables

Variable	Purpose	Default
MONGODB_URI	MongoDB connection string	mongodb://localhost:27017
MONGODB_DB	Database name	gameforge
MONGODB_ALLOW MOCK	Enables/disables mock fallback	enabled unless set to 0

Configuration Notes

- Use local MongoDB for full persistence
- Use mongomock for development/testing without a running database
- Environment-based configuration supports local and cloud deployment

15. Seed Data

The project includes a demo seed script with:

Data Type	Quantity
Users	12
Games	11
Tracked Entries	11

Seed Data Purpose

- Speeds up demonstrations
- Supports analytics immediately
- Provides realistic examples for screenshots and presentations
- Helps validate recommendation and reporting features

16. Analytics and Reporting

GameForge offers several analytics views.

User Statistics Dashboard

Includes:

- Total hours played
- Average sentiment score
- Number of active, completed, and abandoned games
- Top genres or preferences
- Aggregated user gameplay behavior

Financial Analysis Report

Displays:

- Game title
- Price paid
- Hours played
- Cost per hour
- Verdict label

Dust Detection Report

Displays:

- Game title
- Last played date
- Inactive days
- Dust state
- Dust score

Recommendation Results

Displays:

- Suggested game title
- Similarity score
- Shared traits
- Explanation of why it was recommended

17. Strengths and Innovations

Major Strengths

- Flexible schema: supports varied and evolving game metadata
- Multi-dimensional tracking: combines sentiment, time, money, and completion
- Recommendation depth: lore-based discovery goes beyond genre alone
- Backlog intelligence: dust detection supports practical library management
- Financial awareness: cost-per-hour adds real-world decision support
- Offline-first support: mongomock improves development accessibility
- Immersive CLI design: polished terminal UI increases usability and engagement
- Modular codebase: easy to maintain and extend

Innovation Areas

- Emotional sentiment as a tracked metric
- Narrative similarity through lore-linking
- Combined financial and behavioral analytics
- Game backlog management through inactivity scoring

18. Error Handling and Edge Cases

GameForge is designed to handle common failures gracefully.

Covered Cases

- Invalid numeric input
- Duplicate usernames
- Missing or malformed data
- MongoDB connection failure
- Zero-hour financial calculations
- Empty search results
- Missing optional metadata

Example Behaviors

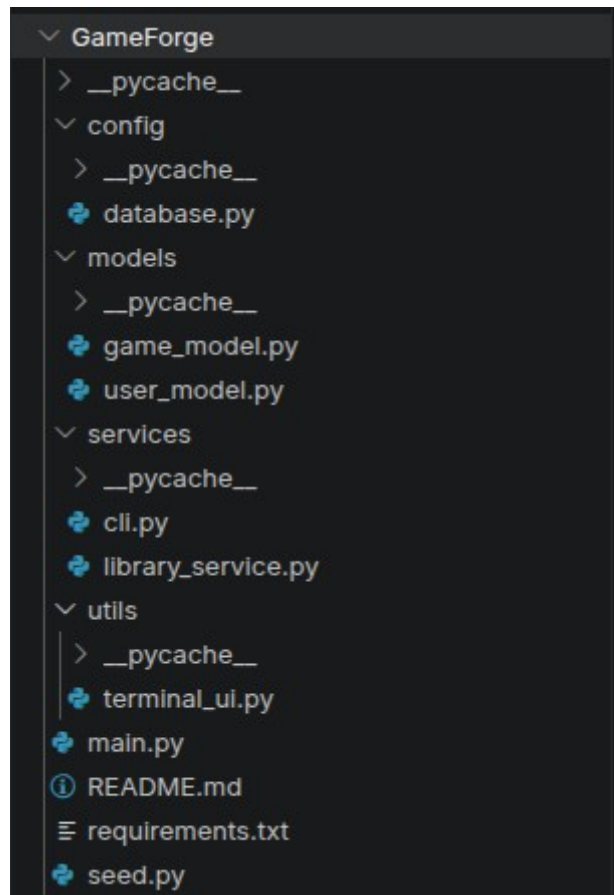
- Prevent crashes on bad input
- Retry prompts for invalid entries
- Warn on duplicate creation attempts
- Fallback to in-memory mock database if MongoDB is unavailable

Example Fallback Message

MongoDB unavailable. Running with in-memory mock database.

19. Screenshots

Directory



main.py

```
main.py x
GameForge > main.py
1  """GameForge console entry point."""
2
3  from services.cli import run_cli
4
5
6  if __name__ == "__main__":
7      run_cli()
8
```

user_model.py

```
main.py user_model.py x
GameForge > models > user_model.py > ...
1  """User data model used by the CLI."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from datetime import datetime, timezone
7  from typing import List, Optional
8
9
10 def utc_now() -> datetime:
11     return datetime.now(timezone.utc)
12
13
14 @dataclass
15 class UserCreate:
16     username: str
17     email: str
18     favorite_platform: str = "Unknown"
19     favorite_genres: Optional[List[str]] = None
20     playstyles: Optional[List[str]] = None
21     bio: str = ""
22
23     def to_document(self) -> dict:
24         now = utc_now()
25         return {
26             "username": self.username.strip(),
27             "email": self.email.strip().lower(),
28             "favorite_platform": self.favorite_platform.strip() or "Unknown",
29             "favorite_genres": self.favorite_genres or [],
30             "playstyles": self.playstyles or [],
31             "bio": self.bio.strip(),
32             "created_at": now,
33             "updated_at": now,
34         }
35
36
37 @dataclass
38 class UserUpdate:
39     username: Optional[str] = None
40     email: Optional[str] = None
41     favorite_platform: Optional[str] = None
42     favorite_genres: Optional[List[str]] = None
43     playstyles: Optional[List[str]] = None
44     bio: Optional[str] = None
45
46     def to_update(self) -> dict:
47         payload = {
48             "username": self.username.strip() if isinstance(self.username, str) else self.username,
49             "email": self.email.strip().lower() if isinstance(self.email, str) else self.email,
50             "favorite_platform": self.favorite_platform.strip() if isinstance(self.favorite_platform, str) else self.favorite_platform,
51             "favorite_genres": self.favorite_genres,
52             "playstyles": self.playstyles,
53             "bio": self.bio.strip() if isinstance(self.bio, str) else self.bio,
54         }
55         return {key: value for key, value in payload.items() if value is not None}
56
```

game_model.py


```
main.py  game_model.py X
GameForge > models > game_model.py > LibraryEntryCreate > to_document
1  """Game and library data models used by the CLI."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass, field
6  from datetime import datetime, timezone
7  from typing import List, Optional
8
9
10 def utc_now() -> datetime:
11     return datetime.now(timezone.utc)
12
13
14 def split_csv(value: str) -> List[str]:
15     return [item.strip() for item in value.split(",") if item.strip()]
16
17
18 SENTIMENT_STATES = ["Completed", "Playing", "Rage-Quit", "Post-Game-Depression"]
19
20
21 @dataclass
22 class GameCreate:
23     title: str
24     genre: List[str]
25     playstyles: List[str]
26     lore_tags: List[str]
27     developer: str
28     release_year: int
29     price_paid: float
30     platform: str = "Unknown"
31     description: str = ""
32
33     def to_document(self) -> dict:
34         now = utc_now()
35         return {
36             "title": self.title.strip(),
37             "genre": self.genre,
38             "playstyles": self.playstyles,
39             "lore_tags": self.lore_tags,
40             "developer": self.developer.strip(),
41             "release_year": self.release_year,
42             "price_paid": self.price_paid,
43             "platform": self.platform.strip() or "Unknown",
44             "description": self.description.strip(),
45             "added_at": now,
46             "created_at": now,
47             "updated_at": now,
48         }
49
50
51 @dataclass
52 class GameUpdate:
53     title: Optional[str] = None
54     genre: Optional[List[str]] = None
55     playstyles: Optional[List[str]] = None
56     lore_tags: Optional[List[str]] = None
57     developer: Optional[str] = None
```

seed data

```
100     UserCreate(  
101         username="VantaPatch",  
102         email="vanta.patch@example.com",  
103         favorite_platform="PC",  
104         favorite_genres=["Action", "Metroidvania", "Adventure"],  
105         playstyles=["Explorer", "Lore Hunter", "Boss Hunter"],  
106         bio="Enjoys hand-crafted worlds and tight progression loops.",  
107     ),  
108     UserCreate(  
109         username="Moonframe",  
110         email="moonframe@example.com",  
111         favorite_platform="PlayStation 5",  
112         favorite_genres=["Adventure", "Narrative", "Horror"],  
113         playstyles=["Story Focused", "Immersion", "Decision Maker"],  
114         bio="Follows emotional stories and atmospheric experiences.",  
115     ),  
116     UserCreate(  
117         username="RiftWarden",  
118         email="rift.warden@example.com",  
119         favorite_platform="PC",  
120         favorite_genres=["MMO", "RPG", "Action"],  
121         playstyles=["Party Leader", "Grinder", "Economy Watcher"],  
122         bio="Likes long-term progression and guild-based play.",  
123     ),  
124     UserCreate(  
125         username="NovaReplay",  
126         email="nova.replay@example.com",  
127         favorite_platform="Nintendo Switch",  
128         favorite_genres=["Roguelike", "Action", "Indie"],  
129         playstyles=["Build Experimentation", "Fast Runs", "Replay Hunter"],  
130         bio="Replays games until every route is mastered.",  
131     ),
```

```
154     GameCreate(  
155         title="Hades",  
156         genre=["Roguelike", "Action", "Indie"],  
157         playstyles=["Fast Runs", "Build Experimentation", "Skill Mastery"],  
158         lore_tags=["Greek Myth", "Underworld", "Narrative Loop"],  
159         developer="Supergiant Games",  
160         release_year=2020,  
161         price_paid=24.99,  
162         platform="PC",  
163         description="Fast-paced escape from the underworld with strong replay value."  
164     ),  
165     GameCreate(  
166         title="The Legend of Zelda: Tears of the Kingdom",  
167         genre=["Adventure", "Action", "Open World"],  
168         playstyles=["Sandbox Creativity", "Puzzle Solving", "Exploration"],  
169         lore_tags=["Sky Islands", "Ganon Return", "Ancient Tech"],  
170         developer="Nintendo",  
171         release_year=2023,  
172         price_paid=69.99,  
173         platform="Nintendo Switch",  
174         description="Large-scale adventure focused on creativity and exploration.",  
175     ),  
176     GameCreate(  
177         title="Neon Circuit",  
178         genre=["Racing", "Arcade", "Action"],  
179         playstyles=["Time Trial", "Precision Driving", "Multiplayer"],  
180         lore_tags=["Neo City", "Street Syndicates", "Turbo Drift"],  
181         developer="Blacklight Forge",  
182         release_year=2024,  
183         price_paid=34.99,  
184         platform="PC",  
185         description="A high-speed street racer in a neon-drenched future city.",  
186     ),
```

```

294     LibraryEntryCreate(
295         user_id=created_users[1]["_id"],
296         game_id=created_games[2]["_id"],
297         hours_played=38.0,
298         status="Playing",
299         sentiment_state="Post-Game-Depression",
300         sentiment_score=4,
301         rating=9.0,
302         comment="Exploring every shrine and side quest.",
303         achievements_unlocked=["Skyview Tower", "Great Sky Island"],
304         completion_percentage=48,
305     ),
306     LibraryEntryCreate(
307         user_id=created_users[2]["_id"],
308         game_id=created_games[3]["_id"],
309         hours_played=22.0,
310         status="Playing",
311         sentiment_state="Rage-Quit",
312         sentiment_score=2,
313         rating=7.5,
314         comment="Kept missing apex corners but the track design is brilliant.",
315         achievements_unlocked=["First Lap", "Night Drift"],
316         completion_percentage=31,
317     ),
318     LibraryEntryCreate(
319         user_id=created_users[4]["_id"],
320         game_id=created_games[4]["_id"],
321         hours_played=57.0,
322         status="Completed",
323         sentiment_state="Completed",
324         sentiment_score=5,
325         rating=9.1,
326         comment="The co-op runs turned chaotic in the best way.",
327         achievements_unlocked=["Final Relay", "Squad Anchor"],
328         completion_percentage=100,
329     ),

```

UI/CLI

```

aditya@oasis:~/Documents/24BEL0044_Aditya$ source /home/aditya/Documents/24BEL0044_Aditya/.venv/bin/activate
(.venv) aditya@oasis:~/Documents/24BEL0044_Aditya$ /home/aditya/Documents/24BEL0044_Aditya/.venv/bin/python /home/aditya/Documents/24BEL0044_Aditya/GameForge/main.py

```



GAMEFORGE

Cyber terminal game tracking system

```

1. Add user
2. List users
3. Update user
4. Delete user
5. Add game
6. List games
7. Update game
8. Delete game
9. Track a game
10. Update tracked game
11. Delete tracked game
12. View user library
13. Inspect game HUD
14. Lore-link suggestions
15. Search games
16. View user stats
17. List all library entries
18. Exit

```

Choose an option: █

Using option 6 to list games:

Choose an option: 6

[GAME VAULT]

◆ ENTRY 1

- ▶ Id: 6a33ab100485e1e0d078ad16
- ▶ Title: Ashfall Protocol
- ▶ Genre: Shooter, Action, Sci-Fi
- ▶ Playstyles: Co-op, Tactical, Loot Chase
- ▶ Lore Tags: Orbital Collapse, Exo Marines, Signal Blackout
- ▶ Developer: Iron Comet
- ▶ Release Year: 2023
- ▶ Price Paid: 49.99
- ▶ Platform: Xbox Series X
- ▶ Description: A co-op tactical shooter about surviving a collapsing orbital colony.
- ▶ Added At: 2026-03-18 08:23:44+00:00
- ▶ Created At: 2026-03-18 08:23:44+00:00
- ▶ Updated At: 2026-03-18 08:23:44+00:00
- ▶ Lore Tag Count: 3
- ▶ Playstyle Count: 3
- ▶ Game Value: RICH
- ▶ Lore Anchor: Orbital Collapse, Exo Marines, Signal Blackout

◆ ENTRY 2

- ▶ Id: 6a33ab100485e1e0d078ad1b
- ▶ Title: Dread Signal
- ▶ Genre: Horror, Adventure, Sci-Fi
- ▶ Playstyles: Immersion, Survival, Narrative
- ▶ Lore Tags: Deep Space, Last Transmission, Echo Chamber
- ▶ Developer: Red Static
- ▶ Release Year: 2023
- ▶ Price Paid: 39.99
- ▶ Platform: PC
- ▶ Description: An atmospheric survival horror game about a dead station still broadcasting.
- ▶ Added At: 2026-04-04 08:23:44+00:00
- ▶ Created At: 2026-04-04 08:23:44+00:00
- ▶ Updated At: 2026-04-04 08:23:44+00:00
- ▶ Lore Tag Count: 3
- ▶ Playstyle Count: 3
- ▶ Game Value: RICH
- ▶ Lore Anchor: Deep Space, Last Transmission, Echo Chamber

◆ ENTRY 3

- ▶ Id: 6a33ab100485e1e0d078ad12
- ▶ Title: Elden Ring
- ▶ Genre: Action, RPG, Souls-like
- ▶ Playstyles: Boss Rush, Exploration, Build Crafting
- ▶ Lore Tags: Shattered Empire, Ancient Gods, Open World Ritual

Checking user 'Moonframe' library

```
10. View user stats
17. List all library entries
18. Exit
```

Choose an option: 12
User ID or username: Moonframe

TRACKED ENTRY 6a33ab100485e1e0d078ad24 ||

```
▶ User: 6a33ab100485e1e0d078ad0e
▶ Game: Forgebound
▶ Hours Played: 73.0
▶ Cost per Hour: $0.27
▶ Financial Verdict: HIGH VALUE
▶ Dust State: FRESH (0 dust days)
▶ Sentiment: Completed / 5
▶ Status: Completed
▶ Completion: 100%
▶ Achievements: Living Metal, Covenant Breaker
▶ Comment: Traversal felt incredible once the weapons clicked.
▶ Last Played: 2025-12-20 08:23:44.906000+00:00
```

Value Score [████████████████████] 98.0%
[DUST] Recently active

Inspecting game 'Neon Circuit'

```
12. View user library
13. Inspect game HUD
14. Lore-link suggestions
15. Search games
16. View user stats
17. List all library entries
18. Exit
```

Choose an option: 13
Game ID or title: Neon Circuit

Neon Circuit ||

```
▶ Title: Neon Circuit
▶ Developer: Blacklight Forge
▶ Platform: PC
▶ Genres: Racing, Arcade, Action
▶ Playstyles: Time Trial, Precision Driving, Multiplayer
▶ Lore Tags: Neo City, Street Syndicates, Turbo Drift
▶ Price Paid: $34.99
▶ Value State: RICH
▶ Added At: 2026-02-28 08:23:44.903000+00:00
▶ Lore Links: 5 candidates
```

Finding lore link suggestion

- 8. Delete game
- 9. Track a game
- 10. Update tracked game
- 11. Delete tracked game
- 12. View user library
- 13. Inspect game HUD
- 14. Lore-link suggestions
- 15. Search games
- 16. View user stats
- 17. List all library entries
- 18. Exit

Choose an option: 14

Game ID or title: Neon Circuit

Lore Linker

- ▶ Source: Neon Circuit
- ▶ 1. The Legend of Zelda: Tears of the Kingdom | Shared Traits: Action
- ▶ 2. Elden Ring | Shared Traits: Action
- ▶ 3. Ashfall Protocol | Shared Traits: Action
- ▶ 4. Hades | Shared Traits: Action
- ▶ 5. Forgebound | Shared Traits: Action

Searching for a game

- 11. Delete tracked game
- 12. View user library
- 13. Inspect game HUD
- 14. Lore-link suggestions
- 15. Search games
- 16. View user stats
- 17. List all library entries
- 18. Exit

Choose an option: 15

Search term: Qua

Quantum Grove

- ▶ Title: Quantum Grove
- ▶ Developer: Moss Circuit
- ▶ Platform: PC
- ▶ Genres: Puzzle, Simulation, Indie
- ▶ Playstyles: Solver, Experimenter, Relaxed
- ▶ Lore Tags: Temporal Trees, Garden Engine, Memory Bloom
- ▶ Price Paid: \$14.99
- ▶ Value State: RICH
- ▶ Added At: 2026-05-22 08:23:44.904000+00:00

Viewing user 'RiftWarden' stats

9. Track a game
10. Update tracked game
11. Delete tracked game
12. View user library
13. Inspect game HUD
14. Lore-link suggestions
15. Search games
16. View user stats
17. List all library entries
18. Exit

Choose an option: 16

User ID or username: RiftWarden

USER STATS

- ▶ Tracked Games: 1
- ▶ Total Playtime: 44.0
- ▶ Avg Rating: 8.5
- ▶ Avg Completion: 63.0
- ▶ Completed Games: 0
- ▶ Total Achievements: 2
- ▶ Total Spent: 27.99
- ▶ Avg Cost Per Hour: 0.6361363636363636
- ▶ Worthwhile Games: 1
- ▶ Dusty Games: 0

Library Completion [] 63.0%

[AVG COST/HR] \$0.64

[DUSTY GAMES] 0

[WORTHWHILE GAMES] 1

MongoDB backend

Documents 11AggregationsSchemaIndexes 5Validation

Type a query: { field: 'value' } or [Generate query](#) ↗

ExplainResetFindOptions ▶

ADD DATA ▼BULK ▼EXPORT DATA ▼EXPORT CODE

25 ▼1 - 11 of 11 ↺ < > ▼☰ {} ▢

```
developer: "Supergiant Games"
release_year: 2020
price_paid: 24.99
platform: "PC"
description: "Fast-paced escape from the underworld with strong replay value."
added_at: 2026-05-04T08:23:44.902+00:00
created_at: 2026-05-04T08:23:44.902+00:00
updated_at: 2026-05-04T08:23:44.902+00:00
```

```
_id: ObjectId('6a33ab1004851e0d078ad14')
title: "The Legend of Zelda: Tears of the Kingdom"
genre: Array (3)
playstyles: Array (3)
lore_tags: Array (3)
developer: "Nintendo"
release_year: 2023
price_paid: 69.99
platform: "Nintendo Switch"
description: "Large-scale adventure focused on creativity and exploration."
added_at: 2026-05-31T08:23:44.903+00:00
created_at: 2026-05-31T08:23:44.903+00:00
updated_at: 2026-05-31T08:23:44.903+00:00
```

```
_id: ObjectId('6a33ab1004851e0d078ad15')
title: "Neon Circuit"
genre: Array (3)
playstyles: Array (3)
lore_tags: Array (3)
developer: "Blacklight Forge"
release_year: 2024
price_paid: 34.99
platform: "PC"
description: "A high-speed street racer in a neon-drenched future city."
added_at: 2026-02-28T08:23:44.903+00:00
created_at: 2026-02-28T08:23:44.903+00:00
updated_at: 2026-02-28T08:23:44.903+00:00
```

vitcampus > gameforge > library

>_ Open MongoDB shell

Documents 11AggregationsSchemaIndexes 3Validation

Type a query: { field: 'value' } or [Generate query](#) ↗

ExplainResetFindOptions ▶

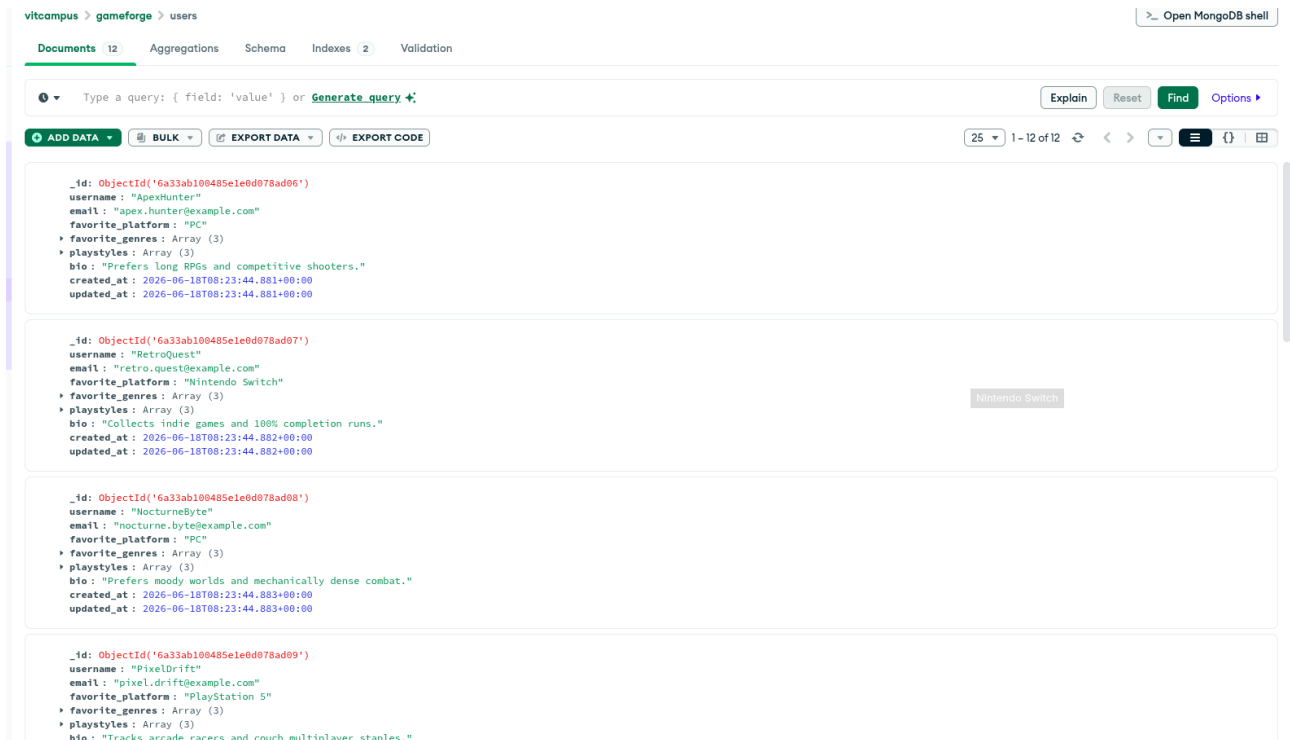
ADD DATA ▼BULK ▼EXPORT DATA ▼EXPORT CODE

25 ▼1 - 11 of 11 ↺ < > ▼☰ {} ▢

```
_id: ObjectId('6a33ab1004851e0d078ad1d')
user_id: "6a33ab1004851e0d078ad06"
game_id: "6a33ab1004851e0d078ad12"
hours_played: 84.5
playtime_hours: 84.5
status: "Playing"
sentiment_state: "Playing"
sentiment_score: 5
rating: 9.5
comment: "Working through late-game bosses."
achievements_unlocked: Array (3)
completion_percentage: 72
last_played_at: 2025-11-10T08:23:44.905+00:00
created_at: 2026-06-18T08:23:44.893+00:00
updated_at: 2025-11-10T08:23:44.905+00:00
```

```
_id: ObjectId('6a33ab1004851e0d078ad1e')
user_id: "6a33ab1004851e0d078ad06"
game_id: "6a33ab1004851e0d078ad13"
hours_played: 61
playtime_hours: 61
status: "Completed"
sentiment_state: "Completed"
sentiment_score: 5
rating: 10
comment: "Finished multiple runs and still returning for more."
achievements_unlocked: Array (2)
completion_percentage: 100
last_played_at: 2026-05-24T08:23:44.905+00:00
created_at: 2026-06-18T08:23:44.893+00:00
updated_at: 2026-05-24T08:23:44.905+00:00
```

```
_id: ObjectId('6a33ab1004851e0d078ad1f')
user_id: "6a33ab1004851e0d078ad07"
game_id: "6a33ab1004851e0d078ad14"
hours_played: 38
playtime_hours: 38
status: "Playing"
```



20. Future Enhancements

Possible extensions include:

- Multi-user cloud sync with MongoDB Atlas
- Achievement systems and milestone badges
- Social sharing and friend recommendations
- Visual analytics and chart dashboards
- Web interface for richer interaction
- Mobile companion app
- Steam/Epic API integration
- Multiplayer and co-op tracking metrics

21. Conclusion

GameForge transforms game library management from a fragmented, passive activity into a unified and insight-driven experience. By combining gameplay tracking, sentiment analysis, financial metrics, backlog detection, and smart recommendations, it provides players with meaningful visibility into both their habits and their collections.

Its modular Python architecture, MongoDB-backed flexible data model, immersive CLI interface, and analytics-focused design make it a distinctive project with both practical value and strong presentation appeal.

22. Appendix

A. Quick Reference Summary

Category	Summary
One-liner	Cyber-themed Python CLI for game library analytics and recommendations
Operations	20
Collections	3
Main analytics	Sentiment, cost/hour, dust score, recommendations
Demo data	12 users, 11 games, 11 entries
Stack	Python 3.10+, MongoDB, pymongo, mongomock

B. Core Formula Summary

Cost per hour

$CPH = \text{hours_played} / \text{price_paid}$

Dust threshold

$\text{dust_state} = \text{DUSTY}$ if $\text{days_inactive} > 180$

Similarity score

$\text{similarity_score} = \text{total_traits} / \text{shared_traits}$